

COMS 3003A

Test 3 – Deferred version

22 May, 2024

This is a closed book test.

Time allowance: 60 minutes

Total number of points: 50

1. For each of the following formulas, find a model where the formula is true and a model where the formula is false:

(a) **(6 points)** $\forall x \forall y (R(x, y) \rightarrow \exists z R(x, z) \wedge R(z, y))$;

True: $M = \langle D, I \rangle$, where $D = \{a\}$ and $I(R) = \{\langle a, a \rangle\}$.

False: $M = \langle D, I \rangle$, where $D = \{a, b\}$ and $I(R) = \{\langle a, b \rangle\}$.

(b) **(6 points)** $\forall x \forall y \forall z (R(x, y) \wedge R(y, z) \rightarrow R(x, z)) \wedge \forall x \exists y R(y, x) \wedge \forall x R(x, x)$.

True: $M = \langle D, I \rangle$, where $D = \{a\}$ and $I(R) = \{\langle a, a \rangle\}$.

False: $M = \langle D, I \rangle$, where $D = \{a\}$ and $I(R) = \emptyset$.

2. **(10 points)** Prove that the formula $\forall x (P(x) \rightarrow Q(c)) \rightarrow (\exists x P(x) \rightarrow Q(c))$ is valid.

Suppose otherwise. Then, there exists a model $M = \langle D, I \rangle$ and an assignment α such that

$$(1) \quad M \not\models \forall x (P(x) \rightarrow Q(c)) \rightarrow (\exists x P(x) \rightarrow Q(c))[\alpha].$$

Then,

$$(2) \quad M \models \forall x (P(x) \rightarrow Q(c))[\alpha];$$

$$(3) \quad M \not\models (\exists x P(x) \rightarrow Q(c))[\alpha].$$

By (3),

$$(4) \quad M \models \exists x P(x)[\alpha];$$

(5) $M \not\models Q(c)$.

By (4), there exists $\alpha' \stackrel{x}{=} \alpha$ such that

(6) $M \models P(x)[\alpha']$.

Since $\alpha' \stackrel{x}{=} \alpha$, by (2), we obtain

(7) $M \models P(x) \rightarrow Q(c)[\alpha']$, and hence

either

(7a) $M \not\models P(x)[\alpha']$, and hence

or

(7b) $M \models Q(c)$.

We see that both cases result in contradiction: (7a) contradicts (6), while (7b) contradicts (5).

3. **(6 points) Prove that the formula $\forall x \exists y R(x, y) \rightarrow \forall x R(x, x)$ is not valid.**

Consider the model $M = \langle D, I \rangle$ where $D = \{a, b\}$ and $I(R) = \{\langle a, b \rangle, \langle b, a \rangle\}$. Let α be an arbitrary assignment in M . Then, $M \models \forall x \exists y R(x, y)[\alpha]$ and $M \not\models \forall x R(x, x)[\alpha]$; hence, $M \not\models \forall x \exists y R(x, y) \rightarrow \forall x R(x, x)[\alpha]$.

4. In the proof of undecidability of the first-order logic, we describe computations of Turing machines on the empty string ϵ . When describing a Turing machine $M = (Q, \Sigma, \Gamma, q_0, q_1, q_2, \delta)$, we use the following vocabulary (remember that every first-order language contains the equality symbol $=$):

- the constant symbol 0 ('zero');
- the unary function symbol S ('successor' on natural numbers);
- for every $k \in \{0, \dots, |Q|\}$, a unary predicate letter $Q_k(x)$ ('at step x , the machine M is in state q_k ');
- a binary predicate letter $C(x, y)$ ('at step x , the machine M scans cell y ');
- for every $k \in \{0, \dots, |\Gamma|\}$, a binary predicate letter $S_k(x, y)$ ('at step x , cell y contains symbol s_k ').

When describing computations of Turing machines, we will be making the following assumptions:

- the tape is one-way infinite (hence the cells can be numbered 0, 1, 2, etc.);
- machines can move left and right (no other types of head movement are allowed);
- the left-most cell, i.e., cell 0, always contains an end-of-tape marker, which is a special tape symbol;
- states are always called q_0, q_1, q_2 , etc.;
- the starting state is q_0 , the accepting state is q_1 , and the rejecting state is q_2 ;
- the tape symbols are always called s_0, s_1, s_2 , etc.
- s_0 is the blank;
- s_1 is the end-of-tape marker.

Write formulas saying the following, in the vocabulary listed above:

- (a) **(4 points)** At every step of the computation, M is in exactly one state.
- (b) **(4 points)** A formula describing the initial configuration of M on the empty word ϵ .
- (c) **(5 points)** Suppose that $q_i s_k \rightarrow q_j s_l L$ is an instruction from δ . Write a formula describing what happens when M executes this instruction.

5. **(9 points)** Recall that a first-order formula φ logically follows from a set Γ of first-order formulas if φ is true in every model where all members of Γ are simultaneously true. Prove that determining logical consequence is undecidable. You may assume the following problems to be undecidable:

- self-acceptance for Turing machines;
- halting for Turing machines;
- halting on the empty string for Turing machines.

You may need to do more than one reduction.

First, we reduce halting on the empty string for Turing machines to first-order validity, as in done in lectures.

Second, we reduce validity to logical consequence. For this, we will use the following observation: for every formula φ ,

$$\models \varphi \iff \emptyset \models \varphi. \quad (*)$$

Now, suppose we have a TM M_C that decides logical consequence. We use M_C to define the TM M_V that solves validity. On input φ , the machine M_V prepares the string $\langle \emptyset, \varphi \rangle$ and then runs M_C on $\langle \emptyset, \varphi \rangle$. If M_C accepts $\langle \emptyset, \varphi \rangle$, then M_V accepts φ ; otherwise, it rejects.

Let's check that M_V indeed decides validity:

If $M_V(\varphi) = 1$, then $M_C(\langle \emptyset, \varphi \rangle) = 1$, which means that $\emptyset \models \varphi$, and so, by (*), $\models \varphi$.

If $M_V(\varphi) = 0$, then $M_C(\langle \emptyset, \varphi \rangle) = 0$, which means that $\emptyset \not\models \varphi$, and so, by (*), $\not\models \varphi$.